GitHub 仓库链接：https://github.com/c2h5ohjiujing-eng/data-Project2

当代人工智能实验二：搜索算法

姓名：李思彤　学号：10242150454

1 目标、实现与实验设置

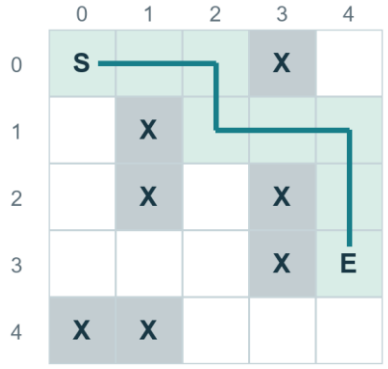
完成迷宫寻路、加权地形、极大极小博弈树和竞争宝藏分配四题；在给定样例正确的基础上，比较启发函数、访问顺序、搜索深度和缓存的影响。代码使用 Python 标准库，题目一、二、三分别返回坐标列表、整数代价和最优得分；题目四输出距离矩阵与决策。

A* 将坐标作为状态，g 为已付代价，h 为剩余代价下界，f=g+h。最小堆保存 (f, tie, serial, g, position)，只在新 g 更小时更新父指针并入堆；弹出时跳过陈旧项，目标出堆后回溯路径。默认同 f 优先较小 h，邻居依次为右、下、左、上，无解返回 [] / -1。

Minimax 在 MAX 层取最大值、MIN 层取最小值；Alpha-beta 维护 MAX 已保证的下界 α 与 MIN 已保证的上界 β，α≥β 时停止搜索剩余兄弟。题三默认严格从左到右访问，叶子读取 value，非叶子读取 children。

设置：Windows 11、CPython 3.12.14；各项预热 1 次、测量 51 次，perf_counter_ns 计时，报告中时间取中位数。A* 的“扩展”不含目标和陈旧出堆；博弈的“访问”含根、叶与缓存命中。博弈计时不含距离预计算，原始记录见 results/。

2 题目一：最短路径如何形成



绿色连线为返回路径，灰色为障碍。

(0, 0) → (0, 1) → (0, 2) → (1, 2)

→ (1, 3) → (1, 4) → (2, 4) → (3, 4)

7 次移动；路径包含起点和终点，共 8 个坐标。

所有路径边均为四邻接，且未穿越 X。

图 1 给定迷宫的最短路径；坐标采用 (row, col)，从 0 开始。

启发函数 / 同 f 规则	扩展数	生成数	中位时间 / μs
h=0 / FIFO	15	17	36.1
曼哈顿 / FIFO	13	16	35.6
曼哈顿 / 较小 h	7	11	26.6

表 1 题一给定迷宫的启发函数与同分策略对照。

四方向单位代价下，曼哈顿距离既不高估剩余步数，也满足一致性。相同 FIFO 条件下，扩展数从 15 降至 13；再改变同 f 规则才降至 7，因此不能把全部节省都归因于启发函数。每条合法路径至少需要 |3-0|+|4-0|=7 步，返回路径达到此下界，故为最短。

3 题目二：进入代价与启发函数

路径选择依据总代价而非步数。样例返回 (0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (3,2) → (4,2) → (4,3) → (4,4)。起点初始 g=0；最后进入 E 的代价按题意为 0，其余七次进入均为 1，所以 8 步的总代价为 7。

位置	(0,0)	(1,0)	(2,0)	(3,0)	(3,1)	(3,2)	(4,2)	(4,3)	(4,4)
进入代价	0	1	1	1	1	1	1	1	0
累计 g	0	1	2	3	4	5	6	7	7

表 2 题二最优路径的逐格进入代价与累计代价。

原始 h=D（曼哈顿距离）不再总可采纳：紧邻 E 的格子有 D=1，但真实剩余代价为 0。实现令 m 为非目标可通行格的最小进入代价，使用 h=m·max(D-1,0)。任意路径至少还要走 D 步，除最后一步外至少 D-1 步各花费 m，因此该式是下界。普通相邻格的 h 最多下降 m，而实际边权不少于 m；进入 E 时 h 已为 0，故仍满足一致性。

若地图含其他 0 代价格，m=0，启发函数安全退化为 Dijkstra；若终点本身非零，则采用全图最小进入代价乘 D。样例在相同 FIFO 下，h=0 与修正 h 的扩展数分别为 14、10，代价均为 7。自设 S→99→E 与下方三个 1 的地图中，程序选择绕路，代价为 3，而非直走的 99。

4 题目三：数值回传与剪枝依据

JSON 样例在最底层回传 5、9、9、2，上层两个 MIN 回传 5、2，根 MAX 因而得到 5。左侧 MIN 在首个子树后已有 β=5；下一个 MAX 刚看到叶子 6，便有 α≥β，剩下的 9 无法改变该 MIN 的选择，于是跳过。共享 max2 对象的 Python 构造也得到 5，统计时按树中出现次数访问。

输入 / 方法 / 顺序	访问数	叶子数	剪枝点	得分
给定树 / Minimax	15	8	0	5
给定树 / AB 左→右	14	7	1	5
给定树 / AB 全层逆序	14	7	1	5
自设树 / Minimax	13	9	0	3
自设树 / AB 左→右	9	5	2	3
自设树 / AB 全层逆序	13	9	0	3

表 3 给定博弈树与自设顺序对照树的搜索开销。

给定树正序、逆序恰好都访问 14 个节点，说明逆序不必然更差。为检验顺序影响，自设 MAX 根的三个 MIN 子树，其叶子依次为 [3,4,5]、[1,2,9]、[0,6,7]。正序较早建立 α=3，后两子树各见到 1、0 即剪枝；全层逆序却延迟有效界的建立，需要遍历全部 13 个节点。

5 题目四：距离预计算与博弈建模

将 A、B、T1…T4 的位置视作 6 个寻路端点，分别运行 A* 得到下表。A、B、宝藏格的进入代价均为 1；经过其他宝藏不收集，已取宝藏格仍通行，另一玩家不阻挡。宝藏价值依次为 9、6、7、7。

起点\终点	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

表 4 6×6 最小移动代价矩阵，行是起点、列是终点；对角线为 0。

状态为 s=(pA,pB,R,SA,SB,turn)：双方当前位置、剩余宝藏元组、双方累计收益和当前玩家。当前玩家选择 t 后，gain=Vt-d(p,t)，只更新其自身位置与得分，移除 t，再交替行动。深度 H 按单次行动计数；H=0 或 R 为空时评价 U=SA-SB，允许负收益，不能因 gain<0 擅自跳过必须执行的回合。

每个首步的精确值如下；为避免把剪枝界误当真值，绘图数据由每个首步独立进行完整 Minimax 得到。两种搜索在两种深度下均与题目预期一致。

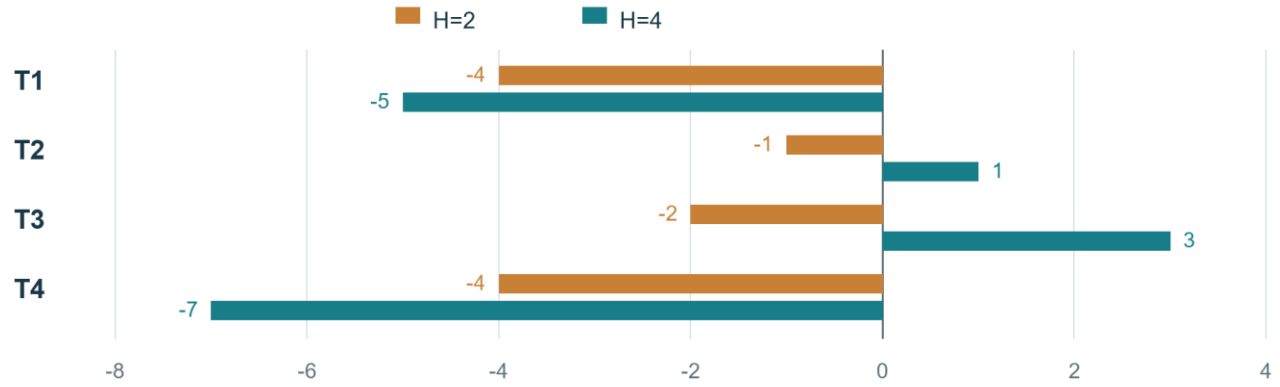


图 2 A 固定首步后，双方最优应对的收益差 U；横轴为收益差。

H=2 的首步值为 [-4,-1,-2,-4]，所以选择 T2；A 从起点到 T2 花费 4，收益 2，B 随后取 T3 收益 3，故 U=-1。此深度只看各走一步，偏好立即获益。

H=4 的首步值变为 [-5,1,3,-7]，最优改为 T3。先取 T3 虽使 A 当下收益为 -1，却占据 B 原本可从起点以收益 3 取得的宝藏，同时改变后续两步的路径代价；完整四回合评价因此优于先取 T2 的 U=1。

当前地图静态、路径中途没有自动收集或碰撞，故只保存端点距离即可保持博弈结果等价。一般“进入格代价”是有向边权，本实现实际计算各有序端点对；样例矩阵对称，是因为各端点的进入代价相同，不能对任意加权图默认对称。

6 搜索深度、访问顺序与对抗行为

下表比较同一地图、同一终局函数下的搜索开销。为固定并列首步的选择，根层一律按 T1…T4；顺序变量只作用于所有内部层。正序为编号顺序，逆序反之，收益排序为当前玩家自身 gain 从高到低。B 应优先尝试自身高 gain 的动作，才能更快压低 U。

H / 方法 / 内部顺序	访问	叶子	剪枝点	时间 / µs
2 / Minimax 正序	17	12	0	34.9
2 / AB 正序	13	8	2	28.4
2 / AB 逆序	15	10	1	32.9
2 / AB 收益排序	13	8	2	32.4
4 / Minimax 正序	65	24	0	139.3
4 / AB 正序	47	16	5	106.7
4 / AB 逆序	51	18	3	120.1
4 / AB 收益排序	43	14	7	118.5

表 5 宝藏分配中搜索深度、算法与内部访问顺序的对照。

H=4 完整树的节点数为 1+4+12+24+24=65。AB 正序访问 47 个，减少 27.7%；收益排序访问 43 个，减少 33.8%。所有方法的首步与得分不变，减少的是不能影响根决策的分支。“剪枝点”是提前停止搜索兄弟节点的次数，不等于跳过的节点总数。

节点减少也不保证耗时减少：本次 H=4 中，AB 正序与收益排序的中位时间分别为 106.7、118.5 µs。此处树很小，额外 gain 计算、排序和函数调用会抵消节点节省；单次微秒级耗时还受调度与缓存影响，因此以确定性的节点数为主、时间为辅。

H=4 的一条最优对抗序列

回合 / 玩家	动作	路费 d	收益 gain	累计 U
1 / A	A → T3	8	-1	-1
2 / B	B → T1	8	1	-2
3 / A	T3 → T2	6	0	-2
4 / B	T1 → T4	12	-5	3

表 6 H=4 的一条最优对抗序列及累计收益差。

终局 SA=-1、SB=-4，因而 U=3。B 最后被迫从 T1 去 T4，花费 12、价值仅 7，收益为 -5。这个例子说明竞争目标是相对收益差，A 的最优策略不一定使自身绝对得分非负，也不一定先拿最近或价值最高的宝藏。这里给出的是一条最优序列，并非声称每个中间选择都唯一。

复杂度：四邻接网格有 E≤4V；在本实现的一致启发函数下，A* 用堆约为 O(V log V)、存储 O(V)。博弈最坏仍为 O(b^H)；良好顺序下 Alpha-beta 可显著减少分支，但不保证最佳情形。若有 n 个宝藏，完整无剪枝树是排列树，叶子为 n!，因此端点预计算、排序和状态复用比单纯提高搜索深度更有实际意义。

7 自设对照、优化与验证

A* 同 f 策略。构造 60×60 空地图，以及障碍率 0.24、种子 20261104 的地图；后者打通顶行和右列以保证存在长 118 的路线。三种方法均得到代价 118，扩展数如下。

地图 / 指标	h=0 / FIFO	曼哈顿 / FIFO	曼哈顿 / 较小 h
空地图：扩展数	3599	3599	118
空地图：时间 / µs	8489.0	8967.9	1110.5
障碍地图：扩展数	2683	1527	118
障碍地图：时间 / µs	5988.4	4185.7	1053.1

表 7 自设大地图上的启发函数与同分策略对照。

空地图上每个单调前进状态都有 f=118，故曼哈顿配 FIFO 仍扩展 3599 个，不能宣称“用了 A* 就一定更快”。同 f 优先较小 h 后只扩展 118 个，但队列峰值由 60 增为 117，节点减少不等于峰值内存同步减少。障碍图预留边界路线且方向顺序有利于该路线，这组结果仅说明机制，不能外推到所有迷宫。

状态缓存与规模拓展。自设 5×5 单位代价地图、6 个宝藏、H=6；位置、价值完整保存在 results.json 的 extension_fixture。全遍历访问 1957 个节点，最优结果为 T6、U=5。缓存键为双方位置、剩余宝藏、轮次与剩余深度，缓存“未来增量 U”而非当前累计 U，避免不同历史得分污染复用；Alpha-beta 另区分 EXACT / LOWER / UPPER 界。

方法	访问	缓存命中	时间 / µs
Minimax	1957	0	4131.3
Minimax + 缓存	997	420	2405.6
AB + 收益排序	409	0	1131.2
AB + 排序 + 缓存	341	65	1095.0

表 8 自设六宝藏场景中的排序与状态缓存对照。

缓存将 Minimax 的重复搜索从 1957 次压到 997 次；AB 排序后为 409 次，再加缓存为 341 次。缓存查表与边界管理也有成本，小型四宝藏问题不保证加速，优化应由规模和重复状态数量决定。

正确性与复现

实际通过 29 项 unittest 测试方法：含 250 张随机迷宫对 BFS、250 张随机加权图对独立 Dijkstra、200 棵随机树对穷举 Minimax，以及 40 个随机宝藏场景的深度、顺序、缓存组合对独立穷举。另测不可达、起终点重合、E=0、零代价退化、绕路优于直走、共享子树、缓存界与累计得分偏移；不是仅比较给定输出。

复现命令（在解压目录执行）：

```
python verify.py
python main.py experiment --repeats 51
python tools/build_report.py
```